
```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
}

```

```

    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

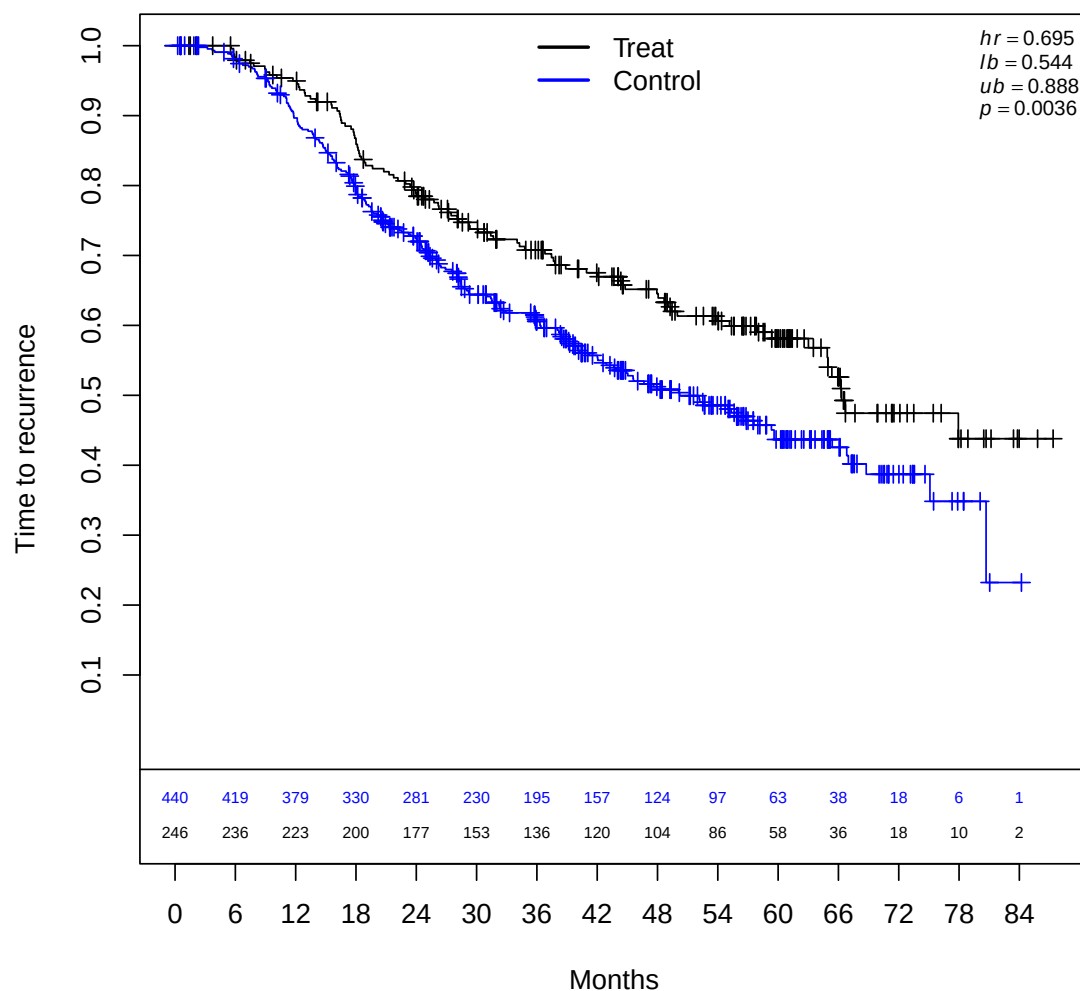
maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2
# out.loc = NULL turns off file creation
N <- 1000
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```
## Super-population empirical harm and non-harm hazard ratios= 2.000007 0.6466405
## Causal HR (empirical ITT)= 0.7057463
```

```
# sim sensH ppvH sensHc ppvHc 1: 11 0.5000000 0.5731707 0.9422442 0.9239482 2:
# 14 0.6891892 0.5666667 0.9376997 0.9622951 3: 17 0.3296703 0.2830189
# 0.8752053 0.8973064 4: 18 0.4489796 0.4943820 0.9252492 0.9116203 5: 20
# 0.8045977 0.4347826 0.8515498 0.9684601 6: 23 0.8101266 0.5203252 0.9049919
# 0.9740035 7: 30 0.7802198 0.5182482 0.8916256 0.9644760

file_out <- c("output/sim-FS4_sim31.Rdata")

n_add_noise <- 3

sim <- 31

dgm <- this.dgm$dgm

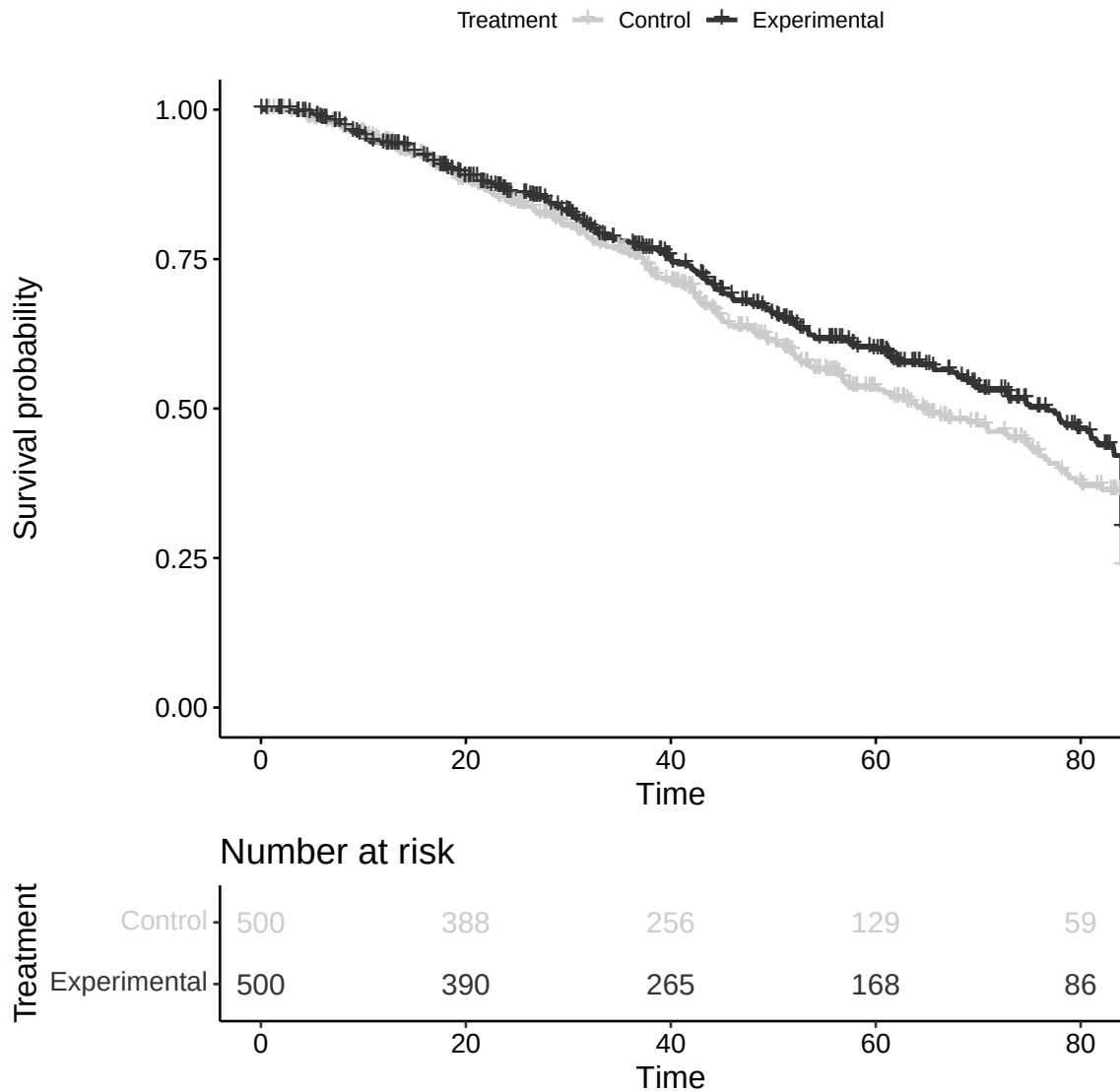
x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)
```

```

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
  legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
  risk.table = TRUE, risk.table.col = "strata")

```



```

if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
  x$noise2 <- rnorm(N)
  x$noise3 <- rnorm(N)
  x$noise4 <- rnorm(N)
  x$noise5 <- rnorm(N)
}

```

```

    confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
                          "noise2", "noise3", "noise4", "noise5")
  }

  if (n_add_noise == 3) {
    set.seed(8316951 + 1000 * sim)
    # Add 3 noise
    x$noise1 <- rnorm(N, sd = 1)
    x$noise2 <- rnorm(N, sd = 1)
    x$noise3 <- rnorm(N, sd = 1)
    confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
                          "noise2", "noise3")
  }

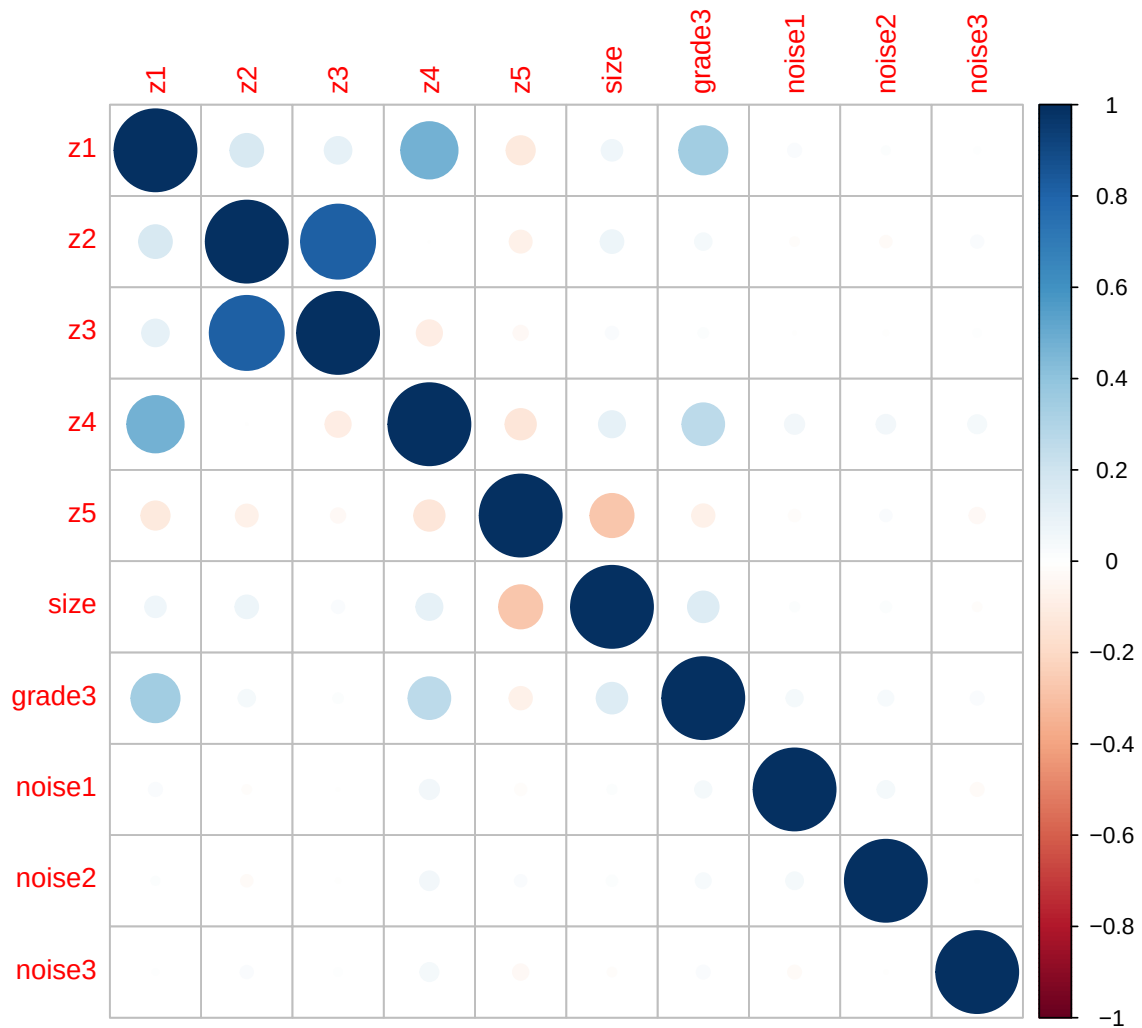
  # More challenging which_replace <- which(confounders.name=='z1')
  # confounders.name[which_replace] <- 'er'

  cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade3+noise1",
                                       "noise2+noise3"))
  coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat  -0.192488  0.824904  0.086376 -2.228 0.025848
## v11     0.360030  1.433373  0.105958  3.398 0.000679
## v21    -0.169226  0.844318  0.148971 -1.136 0.255973
## v31     0.056322  1.057938  0.150861  0.373 0.708897
## v41     0.629020  1.875771  0.101081  6.223 4.88e-10
## v51    -0.922735  0.397431  0.092130 -10.016 < 2e-16
## size   -0.002153  0.997849  0.002836 -0.759 0.447725
## grade3  0.117525  1.124710  0.103329  1.137 0.255379
## noise1 -0.013035  0.987049  0.043192 -0.302 0.762806
## noise2 -0.031559  0.968934  0.042576 -0.741 0.458540
## noise3 -0.036418  0.964237  0.042414 -0.859 0.390552
##
## Likelihood ratio test=226 on 11 df, p=< 2.2e-16
## n= 1000, number of events= 555

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf =T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included

use_lasso <- TRUE
use_grf <- TRUE
use_grf_only <- FALSE

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
```

```

event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

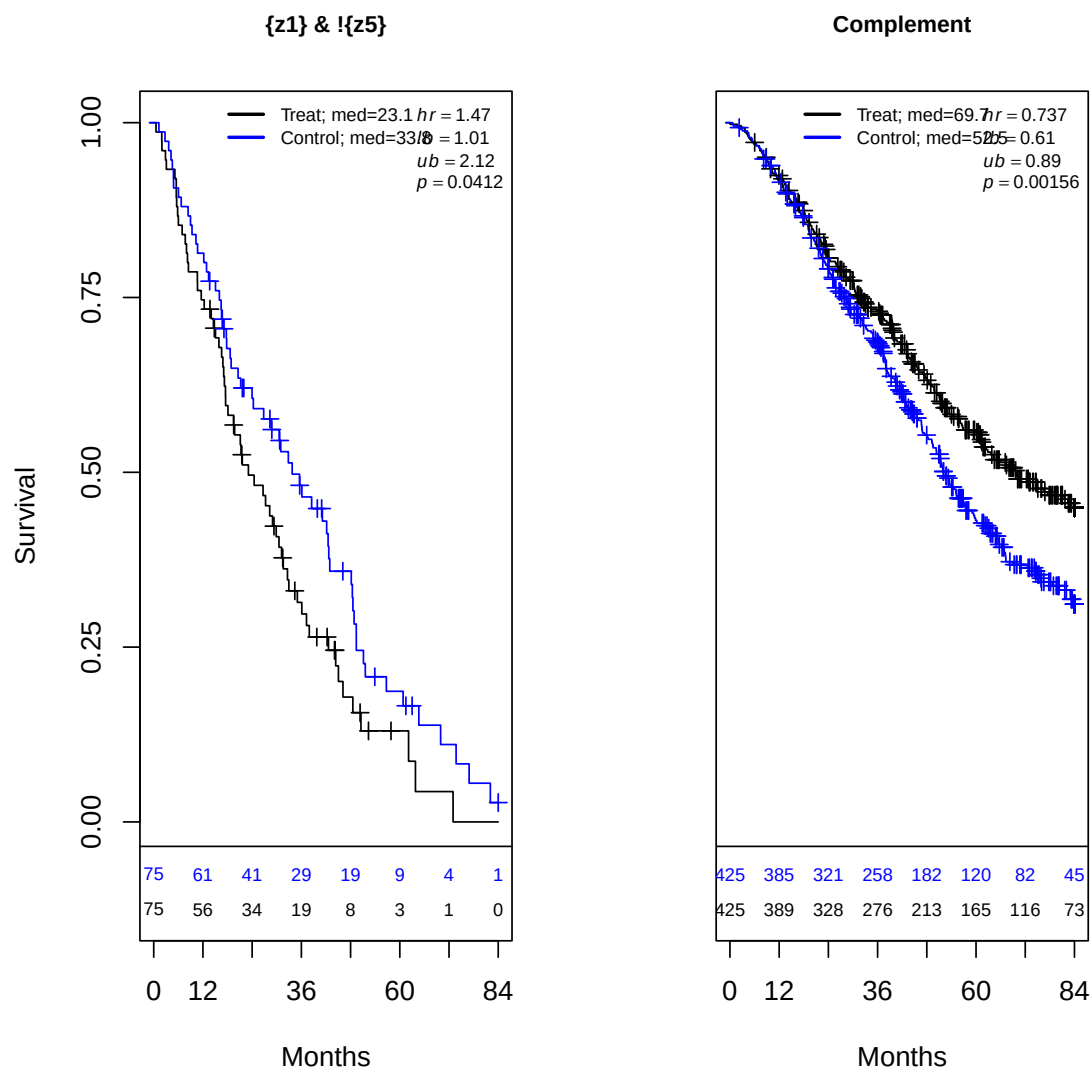
## tau, maxdepth= 49.9606 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2    -3.566880   559.000000   1.258074    3.566880   559.000000
## 2         3     2.971787   441.000000   1.378688   -2.971787   441.000000
## 11        4    -4.111919   542.000000   1.279361    4.111919   542.000000
## 3         6     4.534178   401.000000   1.420526   -4.534178   401.000000
##   treated.se      diff Nsg depth
## 1    1.258074 -7.133761 559     1
## 2    1.378688  5.943575 441     1
## 11   1.279361 -8.223838 542     2
## 3    1.420526  9.068356 401     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 3         6     4.534178   401.000000   1.420526   -4.534178   401.000000
##   treated.se      diff Nsg depth
## 3    1.420526  9.068356 401     2
## NO GRF cuts meeting delta(RMST): dmin.grf= 12
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade3
## 10 x 1 sparse Matrix of class "dgCMatrix"
##               s0
## z1      0.3368616714
## z2     -0.0607765582
## z3      .
## z4      0.5661585955
## z5     -0.8441645446
## size   -0.0001604526
## grade3  0.0739548174
## noise1  .
## noise2  .
## noise3 -0.0054871259
## Cox-LASSO selected: z1 z2 z4 z5 size grade3 noise3
## Cox-LASSO not selected: z3 noise1 noise2
## Median cuts (after Lasso if use_lasso) size noise3
## Categorical (after Lasso if use_lasso) z1 z2 z4 z5 grade3
## Prior to removing (if in GRF): size noise3
## Left (to cut at median) after removing others if in GRF: size noise3
## # of candidate subgroup factors= 7
## [1] "size <= median(size)"      "noise3 <= median(noise3)"
## [3] "z1"                        "z2"
## [5] "z4"                        "z5"
## [7] "grade3"
## Confounders per grf screening q3 q4 q5 q1 q2 q6 q7
##   FSconfounders.name      vi.cs vi_ratio
## 3              q3 0.28757043 2.0129930
## 4              q4 0.20718321 1.4502824
## 5              q5 0.16490680 1.1543476
## 1              q1 0.09518060 0.6662642
## 2              q2 0.09310435 0.6517305
## 6              q6 0.08604764 0.6023335

```

```

## 7          q7 0.06600697 0.4620488
## Number of unique levels (L) and possible subgroups= 14 16383
## # of subgroups based on # variables > k.max and excluded (per million) 0.016278
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 7 8
## # of subgroups with sample size less than criteria 9
## # of subgroups meeting all criteria = 96
## # of subgroups fitted (Cox model estimable) = 96
## *Subgroup Searching Minutes=* 0.00515
## Number of subgroups meeting HR threshold 3
## # of candidate subgroups (meeting HR criteria) = 3
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K    n    E d1    m1    m0    HR L(HR) U(HR) q3.0 q3.1 q4.0 q4.1 q5.0 q5.1
## 1:  2 150 120 61 23.08 33.76 1.47  1.02  2.12    0    1    0    0    0    0
## 2:  2 173 133 78 23.75 42.24 1.46  1.03  2.06    0    1    0    1    0    0
## 3:  2 124  94 46 23.97 38.50 1.29  0.86  1.94    0    1    0    0    0    0
## 4: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 5: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 6: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 7: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 8: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 9: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 10: NA NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
##      q1.0 q1.1 q2.0 q2.1 q6.0 q6.1 q7.0 q7.1
## 1:    0    0    0    0    1    0    0    0
## 2:    0    0    0    0    0    0    0    0
## 3:    0    0    0    0    0    0    0    1
## 4:   NA   NA   NA   NA   NA   NA   NA   NA
## 5:   NA   NA   NA   NA   NA   NA   NA   NA
## 6:   NA   NA   NA   NA   NA   NA   NA   NA
## 7:   NA   NA   NA   NA   NA   NA   NA   NA
## 8:   NA   NA   NA   NA   NA   NA   NA   NA
## 9:   NA   NA   NA   NA   NA   NA   NA   NA
## 10:  NA   NA   NA   NA   NA   NA   NA   NA
## Consistency 0.9575
## # of splits= 400
## Model, % Consistency Met= {z1} !{z5} 0.9575
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K  M.1  M.2
## 1:      0.9575 150      2      1 2 {z1} !{z5}

```



```
## [1] "{z1}" "!("{z5}"
## % consistency criteria met= 0.9575
## SG focus= hr
## Subgroup Consistency Minutes= 0.02366667
## Subgroup found
## Minutes overall= 0.05973333

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 83.26767 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2    -7.322058   738.000000   1.942452     7.322058   738.000000
## 2         3     4.719104   262.000000   3.241085    -4.719104   262.000000
## 21        5    -8.719158   685.000000   1.998662     8.719158   685.000000
## 3         6   -11.151631   123.000000   4.513512    11.151631   123.000000
## 4         7    18.762991   139.000000   4.297314   -18.762991   139.000000
##   treated.se      diff Nsg depth
```

```
## 1      1.942452 -14.644116 738      1
## 2      3.241085   9.438207 262      1
## 21     1.998662 -17.438316 685      2
## 3      4.513512 -22.303263 123      2
## 4      4.297314  37.525982 139      2
##      leaf.node control.mean control.size control.se treated.mean treated.size
## 4          7      18.762991   139.000000   4.297314  -18.762991   139.000000
##      treated.se      diff Nsg depth
## 4      4.297314 37.52598 139      2
## [1] "z1 <= 0"      "size <= 14" "z3 <= 0"
## [1] "z3 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")
## Minutes for Boots 1000 1.9187

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 1.9187

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
```

```

## Propn bootstrap subgroups found = 0.897
# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmns_search > max.minutes)),
    "\n")

## Number timed out= 0
H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)
##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 1.466969 0.2753467 1.015435 2.119286 1.307954 0.2729269 0.8689095 1.968838
print(Hc_estimates)
##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower
## 1: 0.7368537 0.07112228 0.6098485 0.8903086 0.742244 0.1092211 0.5562788
##           H1_upper
## 1: 0.9903778

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")
## Overall minutes for analysis 2.062333
save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# ITT causal hazard ratio
mean(fs.est$df.est$hlin.ratio)
## [1] 0.8327531
# H Oracle
dfH <- subset(fs.est$df.est, flag.harm == 1)
mean(dfH$hlin.ratio)
## [1] 2.249393
dfH_est <- subset(fs.est$df.est, treat.recommend == 0)
mean(dfH_est$hlin.ratio)
## [1] 1.54738
# Hc Oracle
dfHc <- subset(fs.est$df.est, flag.harm == 0)
mean(dfHc$hlin.ratio)
## [1] 0.6040505
dfHc_est <- subset(fs.est$df.est, treat.recommend == 1)
mean(dfHc_est$hlin.ratio)
## [1] 0.7066424
# dfSens <- subset(fs.est$df.est, flag.harm==1 & treat.recommend==0) sensH <-
# nrow(dfSens)/nrow(dfH)

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT	0.805	0.681	0.951	1000	.
H subgroup estimates					
H_{oracle}	1.911	1.293	2.825	139	.
$\hat{H}$	1.467	1.016	2.118	150	86
$\hat{H}_{bc}$	1.308	0.869	1.969	150	86
H-complement subgroup estimates					
H_{oracle}^c	0.668	0.554	0.806	861	.
$\hat{H}^c$	0.737	0.61	0.89	850	797
$\hat{H}_{bc}^c$	0.742	0.556	0.99	850	797

```
## Confounders evaluated in Forestsearch:
## [1] "size <= median(size)"      "noise3 <= median(noise3)"
## [3] "z1"                        "z2"
## [5] "z4"                        "z5"
## [7] "grade3"
```